Software Visualization: Channel Capacity, Space, and Machines

krse

Abstract

This paper examines software visualization from the perspective of information channel capacity. We begin by clarifying what is actually transmitted through the channel—always data, from which the receiver constructs information and knowledge. We formalize this process using van Wijk's model and analyze the visual channel as the highest-capacity human sense. We then compare 2D and 3D space not statically (how much information they contain), but dynamically—how quickly data can be extracted from them. We show that 3D space offers rotation as an additive exploration operation, which unlike zoom does not incur context loss when acquiring new data. Open questions are directed toward machine processing of software: if spatial representation carries data that text cannot transmit in finite time, then multimodal AI systems have access to a fundamentally richer source.

1 What Is Transmitted Through the Channel

Software visualization is ultimately a problem of transmission through a channel with finite capacity. Before comparing channels, we must clarify what is actually transmitted through them.

1.1 Hierarchy: Data, Information, Knowledge

Data are raw symbols without context—lines of source code, pixels on a screen, records in a database. **Information** arises when the receiver assigns context and meaning to data—through ordering, filtering, contextualization [6]. **Knowledge** is internalized information in the mind of a subject; knowledge is non-transferable and must be built through the cognition of information [5]. If information is not interesting to the subject or falls below their attention threshold, it cannot become knowledge.

Shannon entropy quantifies information content:

$$H(X) = -\sum_{i=1}^{n} p(x_i) \log_2 p(x_i) \tag{1}$$

1.2 The Channel Always Transmits Data

A crucial clarification: **the channel always transmits only data**—signals, bits, pixels, phonemes. The channel is indifferent to what these signals mean. What arises from them on the receiver's side depends on the receiver itself:

Phase	What arises	Where it arises	Condition
In the channel	data (signals)	channel	—
After decoding	information	receiver	receiver understands the code / language
After integration	knowledge	receiver	receiver can integrate with K_0

The same signal in the channel may be mere noise for one receiver (does not understand the code), information for another (understands but it is new), and confirmation of existing knowledge for a third. This is a direct consequence of van Wijk's model—perception P depends on K.

Consequence for visualization: when comparing channels (text vs. image, 2D vs. 3D), we compare their ability to **transmit data** from which the receiver can construct information and knowledge. A better channel is one that transmits more relevant data per unit time, not one that "transmits knowledge" (no channel does that).

2 Van Wijk's Model: Visualization as a Feedback System

Van Wijk (2005) [9] formalized visualization as a closed loop between data, tool, and receiver:

$$I(t) = V(D, S, t) \qquad \frac{dK}{dt} = P(I, K) \qquad \frac{dS}{dt} = E(K) \tag{2}$$

where D is data, S the visualization specification, I the resulting image (data in the channel), K the receiver's knowledge, P the perception function, and E the exploration function. Knowledge is an integral:

$$K(t) = K_0 + \int_0^t P(I, K, t)\, dt \tag{3}$$

Three fundamental consequences:

1. **Dependence on K_0.** An experienced programmer constructs more knowledge from the same data (the same image) than a novice. The function P is nonlinear—same data, different knowledge.

2. **Feedback.** The receiver actively modifies the visualization specification (S) based on what has been learned. Visualization is not a one-way transmission; it is a dialogue between the receiver and the data.

3. **Negative knowledge ($\Delta K < 0$).** A poorly designed visualization actively harms—it creates the illusion of patterns where none exist. The data in the channel are technically correct, but the receiver constructs incorrect information and subsequently erroneous knowledge.

3 The Visual Channel: Why the Eye

If the channel transmits data and we want to maximize the volume of data transmitted per unit time, the choice of channel is not arbitrary.

The human visual system has a throughput of ∼10 Mbit/s [4], which is orders of magnitude more than hearing (∼100 kbit/s) or touch (∼1 Mbit/s). Processing occurs in two phases [10]:

- **Phase 1**, parallel, preattentive (< 250 ms): detection of color, orientation, texture, motion, depth. Bottom-up, data-driven.
- **Phase 2**, serial: object identification, engagement of working and long-term memory. Top-down, driven by existing knowledge.

Cleveland and McGill [3] showed that spatial positions are the most accurate way of perceiving quantitative values. Bertin [1] defined six visual variables—size, value, texture, color, orientation, shape—as the fundamental encoding apparatus.

The visual channel is optimal for human receivers in terms of data throughput. This is not a claim about quality of interpretation; it is a claim about **bandwidth**.

4 2D and 3D Space as Dynamic Data Sources

4.1 Static View: 3D Promises More Than It Confirms

Geometrically, 3D space offers more dimensions for encoding data. However, in practice:

$$I_{3D} = C \cdot I_{2D}, \quad \text{where } C < 2 \tag{4}$$

We perceive surfaces, not volumes. Stereo viewing increases data volume by at most less than twofold [11]. Moreover, most experiments test pseudo-3D (projection onto a 2D display), not true 3D with immersion, and therefore conclusions are unjustifiably generalized.

This is, however, a **static** view—how much data fits into a single image. The more interesting question is dynamic.

4.2 Dynamic View: Rate of Data Extraction Over Time

Both spaces, 2D and 3D, are **continuous**. Data are extracted from them through dynamic exploration over time: panning, zoom, and in the case of 3D also rotation.

Exploration space:

- 2D: $\mathcal{E}_{2D} = \mathbb{R}^2 \times \mathbb{R}^+$, position (x, y) + scale s, **three** continuous parameters.
- 3D: $\mathcal{E}_{3D} = \mathbb{R}^3 \times \mathbb{R}^+ \times S^2$, position + scale + viewing angle θ, **six** continuous parameters.

Total volume of data in the channel (image I in van Wijk's model, Section 2) acquired over time T. We use the symbol I consistently with van Wijk—I here denotes the image (data in the channel), not information (that is constructed by the receiver):

$$I_{2D}(T) = \int_0^T \left(\frac{\partial I}{\partial \mathbf{x}} \cdot \dot{\mathbf{x}} + \frac{\partial I}{\partial s} \cdot \dot{s} \right) dt \tag{5}$$

$$I_{3D}(T) = \int_0^T \left(\frac{\partial I}{\partial \mathbf{x}} \cdot \dot{\mathbf{x}} + \frac{\partial I}{\partial s} \cdot \dot{s} + \frac{\partial I}{\partial \theta} \cdot \dot{\theta} \right) dt \tag{6}$$

where $\frac{\partial I}{\partial \mathbf{x}}$, $\frac{\partial I}{\partial s}$, $\frac{\partial I}{\partial \theta}$ are changes in the image upon changes in position, scale, and angle. The difference:

$$\Delta I(T) = \int_0^T \frac{\partial I}{\partial \theta} \cdot \dot{\theta}\, dt \tag{7}$$

4.3 Rotation vs. Zoom: Additive vs. Exchange Gain

Both operations reveal new data. But their character is fundamentally different:

Zoom (common to 2D and 3D):

- Magnification reveals finer details but loses context. Reduction does the opposite.
- Zoom is a **trade-off**: $\frac{\partial I_{\text{detail}}}{\partial s} > 0$, but $\frac{\partial I_{\text{context}}}{\partial s} < 0$. Net gain may be small.
- Zoom has **diminishing returns**; repeated magnification reveals increasingly less informative details.

Rotation (specific to 3D):

- Changing the angle reveals **qualitatively new data**: previously occluded surfaces, relationships, structures.
- Rotation **does not change scale**. The receiver does not gain more detail nor lose context—they gain a different perspective at the same resolution.
- Rotation is not purely additive: uncovering nodes A, B from a new angle may occlude nodes C, D that were previously visible. The net gain $\frac{\partial I}{\partial \theta}$ is the **difference between uncovered and newly occluded** data. For most viewing angles, this difference is positive, because a new perspective reveals qualitatively different structures (different clusters, different connections), not merely mirror-symmetric swaps.
- The key difference from zoom: even though rotation relocates occlusion, **it does not change scale and does not lose context**. Zoom always loses context. Rotation preserves it, even though it changes the visible subset of objects.

If it holds that:

$$\left\langle \frac{\partial D}{\partial \theta} \right\rangle > \left\langle \frac{\partial D}{\partial s} \right\rangle_{\text{net}} \tag{8}$$

then 3D space is a more efficient data source over time, not due to greater static content, but due to **faster extraction** thanks to rotation.

Analogy: zoom is like DFS (depth-first search), rotation is like BFS (breadth-first search). For understanding the global structure of a software system (relationships among 1000 modules), breadth-first exploration is likely more informative.

4.4 Open Questions

1. **Quantification.** Experimental measurement of $\frac{\partial I}{\partial \theta}$ vs. $\frac{\partial I}{\partial s}$ on specific software visualizations has not, to our knowledge, been conducted.

2. **Data type.** For structural data (relationships between components), rotation is likely more efficient; for detailed data (contents of a single class), zoom is more efficient. The optimal strategy is adaptive.

3. **Continuity.** If the information content of continuous rotation demonstrably exceeds the content of a finite set of static views, then interactive rotation is principally superior to static snapshots.

5 Encoding Efficiency: Everything Is Text, but Not All Text Is Equal

5.1 Everything Is Ultimately Read Sequentially

Source code is read character by character. A 2D image bitmap is read pixel by pixel. A 3D scene is rendered into a 2D image and read pixel by pixel. In the deepest sense, **everything is text**—a sequence of symbols from some alphabet, read sequentially.

Source code text has its own alphabet → characters → words → sentences → semantics. When we read "function A calls function B," it is an explicit relationship written in a language with a defined grammar.

A 2D image pixel sequence also has its alphabet (color values), but the "words" and "sentences" of this language are different: **pixel adjacency encodes spatial proximity**, clusters of the same color encode groups, edges encode boundaries. When reading an image pixel by pixel, we read "text"—but text whose grammar is spatial, not linear.

A 3D scene pixel sequence from a given angle is the same "text" as 2D, but **the viewing angle determines which points are visible and which are occluded**. Changing the angle generates new "text"—a new pixel sequence where previously occluded points have become visible. In textual source code encoding, no such operation exists: the text is always the same from any "angle."

5.2 Abstraction Levels of Serialized Data

The fundamental difference between these "texts" is not that they are sequential (they all are), but in the **level of abstraction** at which they were created.

5.2.1 Analogy: Assembly vs. Higher-Level Programming Language

Source code is not a "raw" recording of software. Source code is itself an abstraction—the programmer transformed a mental model into a formal language. The compiler further transforms it into machine code. The result is the same in both cases: instructions for the processor. But the **starting point** has a different level of abstraction.

The same program can be written in assembly or in a higher-level programming language:

- **Assembly (low abstraction):** the programmer must manage registers, addresses, jumps. Every detail is explicit. Expressing a simple idea ("sort the array") requires dozens of instructions. The result is precise but slow to write and to understand.
- **Higher-level language (higher abstraction):** the programmer writes `sort(array)`. Details are hidden. Expressing the same idea requires one line. The result is faster to write and to understand.

Both compile to the same machine instructions. The difference is how much work the programmer (receiver) must do to understand the intent.

5.2.2 Text, 2D, and 3D Encoding: Two Axes, Not One

The analogy with programming languages is useful but incomplete. Software encodings differ along **two independent** axes:

- **Precision**—how unambiguously and exactly the data are encoded.
- **Density**—how many structural relationships are encoded per bit.

Encoding	Precision	Density of structural relationships
Text (edge list)	high (explicit, unambiguous)	low (one relationship per $O(\log n)$ bits)
2D image	lower (implicit, layout-dependent)	high ($O(n^2)$ relationships in one image)
3D + rotation	lower (+ perspective distortion)	highest (new relationships from each angle)

Assembly has high precision and low abstraction. A higher-level language has lower precision (hidden details) and higher abstraction. Likewise, text has high precision and low density; images have lower precision and high density.

The analogy with programming languages holds in that in both cases **higher density/abstraction comes at the cost of lower precision**. It does not hold in that abstraction and density are the same thing—they are not.

Specifically:

Text encoding (high precision, low density): relationships between modules are encoded explicitly but scattered across thousands of lines. The receiver must reconstruct global structure themselves. Analogous to assembly: all details are present, but the overall picture must be assembled.

2D spatial encoding (lower precision, higher density): between the raw system and the pixel sequence, a transformation occurred (layout algorithm) that encoded semantic relationships into spatial positions. The system's structure **is already encoded**; the receiver does not need to reconstruct it. But implicit relations depend on layout quality and are not unambiguous.

3D with rotation (lowest precision, highest density): in addition to the spatial layout, changing the viewing angle generates new views uncovering previously occluded structures. Each view is the result of the same transformation, but from a different perspective. Density is highest, but perspective distortion is added.

5.2.3 Abstraction Adds Noise: The Cost of Hiding Details

Increased abstraction has its cost. Just as a higher-level programming language hides details (the programmer does not know exactly how `sort` is implemented, which may lead to unexpected behavior), spatial encoding hides local software details (exact class contents, return types, internal logic).

This is noise specific to higher abstraction: **loss of detail in exchange for overview**. In assembly, every bit is under control. In a higher-level language, it is not. In text encoding, every edge is unambiguous. In spatial encoding, relations are implicit and layout-dependent.

For 3D encoding, this problem is even more pronounced: perspective distortion, occlusion from other angles, potentially misleading visual artifacts. **The question is not whether 3D adds noise—it does. The question is whether the gain from higher abstraction exceeds the loss from noise.**

5.2.4 Historical Evidence: Abstraction Wins Despite Noise

There is an empirical answer to this question—from the history of software engineering itself.

The transition from assembly to higher-level programming languages introduced measurable noise: the programmer lost control over registers, memory, exact instruction ordering. First-generation compilers generated slower code than hand-optimized assembly. Critics argued (justifiably) that higher-level languages hide details, leading to inefficient programs and unexpected behavior.

An important detail: a higher-level programming language compiles **into** assembly (or directly into machine code). The output is the same—processor instructions. A higher-level language is thus not an alternative to assembly; it is a **layer above it**. Everything a higher-level language produces passes through assembly. And despite the fact that this intermediate step adds noise (compiler optimizations are not always optimal, abstraction hides what actually happens at the instruction level), higher-level languages are used today for the **vast majority of tasks**.

Higher-level languages won. Why?

A higher-level programming language introduced not only noise (less optimal code) but also **higher demands on the computing system**—more memory, greater runtime overhead, slower startup. Abstraction is not free: besides noise in the signal, it also requires more powerful infrastructure for processing.

And despite both disadvantages—noise and increased demands—higher-level languages are used for the vast majority of tasks. Because **the time saved by abstraction exceeded both losses**. A programmer who did not have to think about registers had more cognitive capacity for thinking about the algorithm. The results were faster problem solving, better solutions (freedom from details enabled searching for better algorithms), and fewer errors (smaller code volume = smaller error surface).

5.2.5 Formalization: Condition for Abstraction Advantage

Let T_{low} be the time to solve a task at lower abstraction and T_{high} at higher. Let N_{low} be infrastructure demands (computational, cognitive) at lower abstraction and N_{high} at higher. Let S_{low} be noise at lower abstraction and S_{high} at higher.

Higher abstraction is advantageous when the total cost of solution is lower:

$$T_{\text{high}} + \alpha \cdot N_{\text{high}} + \beta \cdot S_{\text{high}} < T_{\text{low}} + \alpha \cdot N_{\text{low}} + \beta \cdot S_{\text{low}} \tag{9}$$

where α is the weight of infrastructure demands and β is the weight of noise. For programming languages, empirically:

- $T_{\text{high}} \ll T_{\text{low}}$ (5–10x faster development)
- $N_{\text{high}} > N_{\text{low}}$ (higher demands on hardware, runtime)
- $S_{\text{high}} > S_{\text{low}}$ (less optimal code, hidden details)

But the first term dominates: the time savings are so large that they exceed both increased demands and noise. In other words:

$$\Delta T = T_{\text{low}} - T_{\text{high}} \gg \alpha \cdot (N_{\text{high}} - N_{\text{low}}) + \beta \cdot (S_{\text{high}} - S_{\text{low}}) \tag{10}$$

Abstraction is advantageous whenever **time savings** dominate the **sum of increased demands and noise**. Historically, this condition has been met for every successful transition to higher abstraction: assembly → C → object-oriented languages → managed languages → declarative languages. In each case, both demands (N) and noise (S) grew, but time savings (ΔT) grew faster.

5.2.6 Open Question: Is the Current Computational Model Sufficient?

All transitions to higher abstraction so far shared one common property: the output was always the same form—sequential machine code for the von Neumann architecture. Assembly, C, Java, Python—everything ultimately compiles to instructions for a processor that sequentially reads and executes operations over linear memory.

If we follow the analogy with spatial encoding to its logical conclusion, a question arises: what if the computational model itself (processor, architecture) is not optimal for processing spatially encoded data? So far we have increased abstraction on the input (higher languages), but the output (machine code) and processor (sequential) have remained essentially the same.

If we assume that 3D spatial encoding brings even higher abstraction than current programming languages, then hand-in-hand with it will come higher noise and higher infrastructure demands. Time savings will likely be even greater (the condition $\Delta T \gg \alpha \cdot \Delta N + \beta \cdot \Delta S$ will hold), but the resulting machine code will be even less efficient. A hierarchy of output code efficiency emerges:

$$\epsilon_{\text{machine code}} > \epsilon_{\text{assembly}} > \epsilon_{\text{higher language}} > \epsilon_{\text{3D}} \tag{11}$$

where ϵ is the performance efficiency of the resulting machine code (this is not the η from the encoding efficiency definition in Section 5.4). Each higher level of abstraction produces less efficient output. At the same time, the opposite hierarchy holds for problem-solving efficiency:

$$T_{\text{machine code}} \gg T_{\text{assembly}} \gg T_{\text{higher language}} \geq T_{\text{3D}} \ (?) \tag{12}$$

For the first three terms, the hierarchy is empirically verified. The placement of T_{3D} is a **hypothesis**, not a verified fact—3D spatial encoding has not yet been systematically used for solving software tasks at a comparable scale. If, however, the analogy with

previous transitions holds, then $T_{3D} < T_{\text{higher language}}$ for tasks of understanding global software structure.

These two hierarchies run in opposite directions. So far, time savings have always won—we accepted less efficient machine code because we solved the problem many times faster.

But if 3D abstraction produced machine code with efficiency ϵ_{3D} so low that its execution on current von Neumann architecture would be impractically slow, then the time savings during development would be negated by time lost during execution. In that case, it is not enough to increase abstraction on the input—the processor on the output must also change. (Note: ϵ denotes performance efficiency of output code, while η in Section 5.4 denotes information efficiency of encoding—these are distinct quantities.)

It may be beneficial to explore computational architectures that natively work with spatial structures, not with linear instruction sequences. Graphics processors (GPUs) are a partial step in this direction, natively processing 2D/3D data in parallel. Neural accelerators (TPU, NPU) are another. But the question of whether there exists a fundamentally different architecture optimized for processing multidimensional, spatially encoded data about software is beyond the scope of this paper. We note it as a direction for future research.

5.2.7 Application to 2D/3D Visualization

The same logic applies to spatial encoding of software:

Noise from abstraction (loss): the receiver loses control over details. Exact class contents, function return types, internal module logic—none of this is directly visible in spatial encoding. Implicit relations depend on layout quality. Perspective distortion in 3D can mislead.

Gain from abstraction (savings): the receiver does not need to reconstruct global structure from thousands of local fragments. The structure is visible directly. Cognitive capacity (in humans) or computational capacity (in machines) that would be spent on reconstruction is freed for higher tasks: identifying problem areas, searching for better architecture, understanding inter-subsystem dependencies.

If the analogy with programming languages holds, then:

- Noise from spatial abstraction (perspective distortion, implicit relations, detail loss) is analogous to slower compiled code—a real but bounded loss.
- Gain from spatial abstraction (direct structural overview, faster comprehension, freed capacity for higher tasks) is analogous to faster development—a potentially manifold gain.
- And just as with programming languages, the optimal solution is not "only assembly" or "only higher-level language," but a **combination**: higher-level language for most work, assembly for critical sections. Analogously: spatial visualization for global structure, text for precise local details.

Section 5.8 formally argues that for complex systems ($m \gg n$), the gain from eliminating occlusion through rotation exceeds the noise from perspective distortion. The historical analogy with programming languages strengthens this argument: the history of software engineering repeatedly shows that abstraction wins despite noise, if the gain/loss ratio is sufficiently high.

All three encodings produce sequential data ("text"), but the quality of that text varies because it was created at different levels of abstraction. Spatial encoding is text through which abstraction has already occurred—and therefore the receiver extracts information from it more efficiently, at the cost of losing control over details. Historical experience suggests that this trade-off is favorable.

5.3 Three Encodings of the Same Object: Formalization

Consider a software system G with n modules and m dependencies (a graph). We want to transmit it to a receiver through a channel with capacity C bits per second.

Text encoding, edge list:

Each edge requires identification of two nodes: $O(\log n)$ bits per node, thus $O(\log n)$ per edge. Total volume:

$$B_{\text{text}} = O(m \log n) \text{ bits} \tag{13}$$

Text encodes **explicitly** each local relationship (edge A→B). Global structure (clusters, symmetries, graph shape) is **not encoded at all**; the receiver must reconstruct it by processing all m edges.

2D image encoding, spatial graph layout:

An image has $w \times h$ pixels, c bits per pixel:

$$B_{\text{2D}} = O(w \cdot h \cdot c) \text{ bits} \tag{14}$$

But what do these bits encode? The position of each pixel is a **coordinate**, and the mutual position of pixels encodes **spatial proximity**. If the layout algorithm placed semantically related modules near each other, then spatial proximity in the image = semantic relatedness in the system. This is **implicit encoding of relations**; the receiver perceives them without having to read an edge list.

How many relations does a single image encode? Each node has a position, so for n nodes we have $O(n^2)$ mutual distances, all encoded simultaneously in a single image. In text, explicitly describing $O(n^2)$ distances would require $O(n^2 \log n)$ bits.

3D encoding with rotation, series of views:

A single view: $O(w \cdot h \cdot c)$ bits (same as 2D). But changing the angle by $d\theta$ requires only $O(1)$ bits of parameter (new angle)—and generates a **completely new** image of $w \times h \times c$ bits that may reveal:

- previously occluded nodes and edges,
- a new spatial configuration (different clusters emerge),
- new implicit relations.

Marginal bit-cost of angle change: $O(1)$ for the angle parameter, but in practice the receiver gets a new raster of $O(w \cdot h \cdot c)$ bits. For small $d\theta$, however, most of the frame is identical to the previous one—differential encoding (analogous to a video codec) transmits only changes, so the actual marginal cost is $O(\delta \cdot w \cdot h \cdot c)$ where $\delta \ll 1$ is the fraction of changed pixels. Marginal information gain: potentially $O(n)$ new relations (uncovered nodes and edges).

5.4 Efficiency of Encoding Relational Information

To compare encodings, we define **encoding efficiency** η as the ratio of semantically relevant relations R to the volume of transmitted data B:

$$\eta = \frac{R}{B} \tag{15}$$

This is a simplified metric measuring relation density per bit. It relates to data quality Q (defined in Section 6.1) in that higher η means higher $I_{\max}^{\text{enc}}$ at equal B, and thus higher Q. We use η for relation density and Q for overall data quality including noise.

Encoding	Transmitted bits B	Encoded relations R	Efficiency η
Text (edge list)	$m \log n$	m explicit	$\frac{1}{\log n}$
2D image (1 view)	$w \cdot h \cdot c$	$O(n^2)$ implicit (distances)	$\frac{n^2}{w \cdot h \cdot c}$
3D rotation ($d\theta$)	$O(\delta \cdot w \cdot h \cdot c)$ differential, $\delta \ll 1$	$O(n)$ new (uncovered)	$\frac{n}{\delta \cdot w \cdot h \cdot c}$

The marginal efficiency of 3D rotation is $O(n/(\delta \cdot w \cdot h \cdot c))$. For small δ (smooth rotation, most of the frame unchanged), this number is large. The marginal efficiency of text is $O(1/\log n)$, decreasing with system size. For large systems, the gap widens.

For large software systems ($n \to \infty$), the gap widens: spatial encoding is **asymptotically more efficient** than text encoding for relational information.

5.5 Caveats and Limits

This argument has clear limitations:

1. **Applies to relational/structural information.** For precise, local information (variable name, function return type), text is more efficient: $O(\log |\Sigma|)$ bits per character, no ambiguity. An image cannot encode "this function returns `int`" more precisely than text.

2. **Depends on layout quality.** Implicit encoding of relations in the image works only when the layout algorithm correctly maps semantic proximity to spatial proximity. Bad layout = noise, not information. In van Wijk's terminology: the specification S must be of high quality.

3. **Transmission is discrete.** In practice, rotation is not continuous; the display has finite resolution and frame rate. The argument about $O(1)$ bits per angle change is an idealization. In reality, each new view costs $w \cdot h \cdot c$ bits. However, differences between consecutive frames are small (coherence) and can be encoded differentially; a video codec transmits only changes, not entire frames. The marginal cost of rotation is therefore in practice much less than a full frame.

4. **Implicit relations require a capable receiver.** Text is explicit: edge A→B is unambiguous. Spatial proximity is implicit; the receiver must have the ability to interpret it (preattentive processing in humans, convolutional layers in machines). If the receiver lacks this ability, implicit relations are lost.

5.6 Formalization: When Is Spatial Encoding Provably Better?

We can formulate the condition under which spatial encoding is more efficient than text:

Let $G = (V, E)$ be a graph with $|V| = n$ and $|E| = m$. Let R_{struct} be the number of structural relations (clusters, symmetries, topological properties) relevant for understanding the system. Then:

- Text requires at least $\Omega(R_{\text{struct}} \cdot \log n)$ bits for their explicit description.
- A single 2D image encodes them implicitly in $O(w \cdot h \cdot c)$ bits, if the layout is semantically correct.
- 3D rotation uncovers additional structural relations at marginal cost.

Spatial encoding is more efficient when:

$$R_{\text{struct}} \cdot \log n \gg w \cdot h \cdot c \tag{16}$$

i.e., when there are many structural relations (complex system) and image resolution is appropriate. For $n = 1000$, $m = 5000$, a typical display $1920 \times 1080 \times 24$:

- R_{struct} for a graph with 1000 nodes: on the order of $O(n^2) = O(10^6)$ pairwise distances
- Text: $10^6 \cdot 10 \approx 10^7$ bits
- Image: $1920 \cdot 1080 \cdot 24 \approx 5 \cdot 10^7$ bits

For a single static view, the condition $R_{\text{struct}} \cdot \log n \gg w \cdot h \cdot c$ **is not met**: the image ($5 \cdot 10^7$ bits) is actually larger than the text (10^7 bits). Bit volume alone does not argue in favor of the image.

However, bit volume is not the only relevant metric. What matters is the **processing speed by the receiver**: an image is processed in ∼200 ms (preattentively), while text with 10^7 bits takes seconds to minutes. Effective throughput (relations extracted per second) is orders of magnitude higher with the visual channel. And 3D rotation adds further views at marginal differential cost $O(\delta \cdot w \cdot h \cdot c)$, where $\delta \ll 1$.

Here an important distinction is revealed. The condition $R_{\text{struct}} \cdot \log n \gg w \cdot h \cdot c$ compares **bit volume**—a property of data. In this comparison, the image does not win. But Section 5.8 shows that the image has lower encoding entropy ($H_{\text{enc}}^{3D} \leq H_{\text{enc}}^{2D}$) for complex systems—and this is also a property of data, not of the receiver.

If we want to include processing speed, the receiver enters the picture:

$$\frac{R_{\text{struct}}}{B_{\text{image}}/v_{\text{visual}}} > \frac{R_{\text{struct}}}{B_{\text{text}}/v_{\text{textual}}} \tag{17}$$

where v_{visual} and v_{textual} are processing speeds by the receiver. For a human receiver, $v_{\text{visual}}/v_{\text{textual}} \approx 10^2$, which outweighs the bit ratio $B_{\text{image}}/B_{\text{text}} \approx 5$. But this is a property of the receiver, not of the data.

Therefore, the argument for spatial encoding stands on **two legs**:

1. **Property of data (receiver-independent):** lower entropy H_{enc} and higher $I_{\max}^{\text{enc}}$ for spatial encoding (Section 5.8). This holds for any receiver.
2. **Property of receiver (dependent):** higher visual data processing speed in humans. For a machine receiver, this ratio is an open empirical question.

5.7 Continuity of 3D Space and the Radon Transform

A 3D object observed from different angles generates uncountably many 2D projections. The function $\theta \mapsto \pi_\theta(O)$ is continuous on S^2. Research in computed tomography (Radon transform) shows that a 3D object is fully reconstructable from infinitely many 2D projections; the information in the set of all projections is equivalent to the information in the object itself.

In practice, discretization occurs—a finite number of views. But the key question remains: how much information is lost through discretization? If the information content of continuous rotation flow is demonstrably greater than the content of a finite set of snapshots, then every discretization incurs measurable loss, and interactive, smooth rotation is principally superior to static views.

The objection "everything can be displayed in 2D" is mathematically true (space-filling curves), but irrelevant in the dynamic context: a 2D projection loses data (occlusion), reconstruction requires more projections or prior knowledge (K_0), and with increasing complexity, the number of needed views grows—potentially faster than the channel capacity allows transmission.

5.8 Noise, Overload, and Entropy: Why $Q_{2D} \leq Q_{3D}$

So far we have compared the volume and efficiency of useful data transmission. But the channel does not transmit only signal; it also transmits **noise**—data that do not produce information or produce erroneous information. For a correct comparison of encodings, we must also include noise.

5.8.1 Sources of Noise in Individual Encodings

Text: noise in text is relatively low. Text is explicit and unambiguous: edge A→B is either correct or incorrect. The main source of noise is **redundancy and overload**—for large systems, the edge list is so long that the receiver loses the ability to distinguish relevant relationships from irrelevant ones. This is noise not in the data but in the reception process: data volume exceeds the receiver's processing capacity, leading to information overload. Overload reduces effective Q because the receiver extracts less information from B bits than the data potentially contain.

2D image: noise has two components. First is **visual noise**—layout artifacts, edge overlap, misleading groupings caused by the layout algorithm (not by system semantics). Second is **occlusion**—in dense graphs, nodes and edges overlap, becoming unreadable. The receiver must repeatedly zoom, which increases time and thus increases the risk of overload. In 2D, the receiver has no other tool for resolving occlusion than zoom (a trade-off).

3D with rotation: shares visual noise with 2D (depends on layout). But occlusion is resolved by **rotation, not zoom**. This is the key difference: when the receiver in 3D encounters occlusion (nodes behind other nodes), they change the viewing angle and occluded nodes are revealed—without context loss, without scale change, without overload. In 2D, the receiver must zoom (context loss) or pan (overview loss), which increases the total volume of processed data per unit of acquired information.

5.8.2 Encoding Entropy with Respect to Information

Let us define **encoding entropy** H_{enc} as a measure of disorder (noise) in data relative to extracted information:

$$H_{\text{enc}} = \frac{B_{\text{noise}}}{B_{\text{total}}} \tag{18}$$

where B_{noise} are bits that do not contribute to information (redundancy, occlusion, visual artifacts, overload) and B_{total} is the total data volume. Data quality Q (defined in Section 6.1) then depends on encoding entropy: higher H_{enc} means more noise, thus lower Q. Since H_{enc} is a property of data (not of the receiver), comparing H_{enc} between encodings is a receiver-independent quality comparison.

For 2D and 3D encoding of the same system:

Noise source	2D	3D
Visual layout artifacts	present	present (comparable)
Occlusion	present, resolved only by zoom (exchange)	present, resolved by rotation (additive)
Overload from repeated zoom	higher (more zoom-in/out iterations)	lower (rotation does not require repetition)
Perspective distortion	none	present

3D adds perspective distortion (3D-specific noise) but **eliminates occlusion** and **reduces overload** from repeated zoom. For systems with dense topology (many overlapping edges), occlusion is the dominant noise source in 2D—and its elimination through rotation is the fundamental advantage of 3D.

5.8.3 The Inequality $Q_{2D} \leq Q_{3D}$

Argument for $Q_{2D} \leq Q_{3D}$:

1. Both encodings share visual noise from layout (comparable with the same algorithm).
2. 3D adds perspective distortion—its own noise that 2D does not have.
3. But 3D eliminates occlusion through rotation. Occlusion in 2D requires zoom (exchange, increasing overload), while rotation in 3D is additive and does not require repetition.
4. For systems where occlusion is a problem (dense graphs, $m \gg n$), the savings from eliminating occlusion exceed the added noise from perspective distortion.

Therefore, for structurally complex systems ($m \gg n$):

$$H_{\text{enc}}^{3D} \leq H_{\text{enc}}^{2D} \tag{19}$$

Moreover, the maximum extractable information in 3D is at least as large as in 2D, because 3D contains more views of the same object: $I_{\max}^{3D} \geq I_{\max}^{2D}$. Since $Q = (1 - H_{\text{enc}}) \cdot I_{\max}^{\text{enc}}/B$ and at comparable B (same single-frame resolution) both $(1 - H_{\text{enc}}^{3D}) \geq (1 - H_{\text{enc}}^{2D})$ and $I_{\max}^{3D} \geq I_{\max}^{2D}$ hold, we obtain:

$$Q_{3D} \geq Q_{2D} \tag{20}$$

The inequality is strict ($Q_{3D} > Q_{2D}$) when occlusion is the dominant noise source—that is, for dense, complex software systems, which is precisely the case where visualization is most needed.

For simple systems ($m \approx n$, minimal occlusion), $Q_{2D} \approx Q_{3D}$ may hold—3D offers no advantage because the occlusion problem does not exist.

5.8.4 Note on Scope

This paper analyzes **why** 3D encoding should be of higher quality. The question of **how** to correctly (efficiently, with quality) transform 3D spatial representation of software into data—the specific layout algorithm, rendering, perspective selection—is a separate problem beyond the scope of this paper.

6 Data Quality and Consequences for Any Receiver

6.1 Definition of Data Quality

So far we have compared channels from the perspective of volume (how many bits) and speed (how many bits per time). But the key question is different: at **equal data volume** B, how much information can the receiver extract?

Let us define **data quality** Q as the amount of potentially extractable information per bit:

$$Q = \frac{(1 - H_{\text{enc}}) \cdot I_{\text{max}}^{\text{enc}}}{B} \tag{21}$$

where $I_{\text{max}}^{\text{enc}}$ is the maximum information contained in the given encoding (depends on encoding, not receiver), H_{enc} is the noise fraction in data (defined in Section 5.8), and B is the data volume in bits. Data quality is a property of the **encoding**, not the receiver, because both $I_{\text{max}}^{\text{enc}}$ and H_{enc} are properties of the data themselves. A specific receiver extracts $I_{\text{extracted}} \leq (1 - H_{\text{enc}}) \cdot I_{\text{max}}^{\text{enc}}$ depending on their capabilities and K_0, but the **ratio of Q between encodings** is preserved for any receiver capable of processing both forms.

6.2 Why Data Quality Is Receiver-Independent

Previous sections argued by analogy: "if visualization helps humans, perhaps it helps machines too." This is a weak argument, because machines process data differently than humans (they lack preattentive processing, hippocampal navigation, or stereoscopic vision).

The argument from data quality is stronger because it bypasses the question "how does the receiver process":

1. Take B bits of text encoding of graph G (edge list). These bits encode m explicit local relationships. They do not encode global structure.

2. Take B bits of spatial encoding of the same graph G (2D/3D layout). These bits encode node positions, from which follow explicit edges AND implicit spatial relationships ($O(n^2)$ pairwise distances, clusters, symmetries).

3. At equal B, spatial encoding carries **more extractable structure**. This is a fact about data, not about the receiver.

Any receiver capable of processing spatial relationships (whether through preattentive processing, convolutional layers, or any other mechanism) extracts more information from spatial encoding than from text encoding of the same volume. The difference is not in **who** processes, but in **what** is encoded in the data.

6.3 Quality of Three Encodings

For graph G with n nodes and m edges, at fixed volume B bits:

Encoding	What is in B bits explicitly	What is in B bits implicitly	Quality Q
Text	$O(B/\log n)$ edges	nothing	low for structural information
2D image	positions of n nodes, m edges	$O(n^2)$ pairwise distances, clusters	higher
3D + rotation	same as 2D for each view	new relationships from each angle, at marginal ΔB	highest

The key difference: with 3D encoding and rotation, marginal quality $\Delta Q/\Delta B$ is high because a small data change (new angle) uncovers much new structure. With text, marginal quality is consistently low: each new edge adds only one local relationship.

6.4 Consequences: The Argument Holds for Humans and Machines

If $Q_{\text{3D}} > Q_{\text{text}}$ at equal B, then:

1. **For humans:** the visual channel is not just "faster" (higher bandwidth), but transmits **higher-quality data**—data with a higher proportion of extractable structural information per bit.

2. **For machines:** the same argument holds without analogy. A multimodal model receiving B bits of spatial encoding has access to data with higher Q than a model receiving B bits of text. It is irrelevant whether the machine processes the image with convolutions, attention mechanisms, or any other method. What matters is that in those B bits there is **more extractable structure**.

3. **Practical consequence:** if we want an AI system to understand a software system with 1000 modules, we should investigate whether a combination of text (precise local relationships) and spatial visualization (global structure) yields measurably better results than text alone. Not because "machines should see like humans," but because **data in spatial form have higher quality for structural information**.

6.5 Counterarguments

- LLMs work excellently on pure text. There is no empirical evidence that visual input brings improvement. Perhaps Q_{text} is sufficient for current tasks.

- Quality Q depends on layout algorithm quality. Bad layout means $Q_{\text{3D}} < Q_{\text{text}}$—spatial encoding is not automatically better, only potentially better.

- For local, precise information (function name, variable type), $Q_{\text{text}} > Q_{\text{image}}$—text is higher quality for this class of information. The argument for higher spatial encoding quality applies specifically to **structural and relational** information.

- Formal verification requires precision that text provides and images do not.

6.6 Direction: Not "Instead Of" but "In Addition To"

From the definition of data quality, it follows that the optimal approach combines both channels:

- **Text** for local, precise, unambiguous data (Q_{text} is high here).
- **Spatial visualization** for global, structural, relational data (Q_{3D} is high here).

This is not a compromise but a **synergy**—both encodings are high-quality for different classes of information and complement each other.

6.7 Research Agenda

1. **Experimental measurement of Q.** For specific software systems, quantify Q_{text} and Q_{3D} at equal B. Measure the number of correctly identified structural properties (clusters, critical paths, symmetries) by the receiver per unit time.
2. **Receiver-independence test.** If $Q_{\text{3D}} > Q_{\text{text}}$, does it hold for both human and machine receivers? Compare performance: (a) LLM + text, (b) multimodal model + text + visualization, on tasks of identifying global software structure.
3. **Existence proof.** Find a class of software systems where $Q_{\text{3D}}/Q_{\text{text}} \to \infty$ with growing n. According to the encoding efficiency table (Section 5.3), such a class should exist for dense graphs with rich topology.

Conclusion

The channel always transmits data. Information and knowledge arise only in the receiver. But not all data are of equal quality: at equal bit volume B, spatial encoding carries more extractable structural information than text encoding. This is a property of data, not of the receiver, and therefore holds for any receiver, human or machine.

Van Wijk's model shows that knowledge construction from data is nonlinear and subjective. Comparison of exploration integrals shows that 3D space offers rotation—an additive operation that uncovers new data without context loss. The marginal efficiency of 3D rotation is $O(n/(\delta \cdot w \cdot h \cdot c))$, where $\delta \ll 1$, while text efficiency is $O(1/\log n)$—for large systems, the gap widens.

The definition of data quality $Q = (1 - H_{\text{enc}}) \cdot I_{\text{max}}^{\text{enc}}/B$ enables comparing encodings without dependence on receiver type. If $Q_{\text{3D}} > Q_{\text{text}}$ for structural information (as the efficiency table suggests), then multimodal AI systems receiving text and spatial visualization have access to a higher-quality data source. Not because they "should see like humans," but because spatial encoding simply contains more per bit for structural information. Empirical validation of this hypothesis is open and testable.

References

[1] Bertin, J. (1967). *Sémiologie Graphique*. Gauthier-Villars.

[2] Card, S. K., Mackinlay, J. D., & Shneiderman, B. (1999). *Readings in Information Visualization: Using Vision to Think*. Morgan Kaufmann.

[3] Cleveland, W. S., & McGill, R. (1984). Graphical perception: Theory, experimentation, and application to the development of graphical methods. *Journal of the American Statistical Association*, 79(387), 531–554.

[4] Koch, K., et al. (2006). How much the eye tells the brain. *Current Biology*, 16(14), 1428–1434.

[5] Pritchard, D. (2010). Knowledge and Understanding. *The Nature and Value of Knowledge*. Oxford University Press.

[6] Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal*, 27(3), 379–423.

[7] Šesták, K. (2015). *Utilization of visual representation of models in post-project phases of the software systems life cycle*. Doctoral dissertation, TUKE FEI.

[8] Teyseyre, A. R., & Campo, M. R. (2009). An overview of 3D software visualization. *IEEE Transactions on Visualization and Computer Graphics*, 15(1), 87–105.

[9] van Wijk, J. J. (2005). The value of visualization. *Proceedings of IEEE Visualization*, 79–86.

[10] Ware, C. (2004). *Information Visualization: Perception for Design*. 2nd ed. Morgan Kaufmann.

[11] Ware, C., & Franck, G. (1996). Evaluating stereo and motion cues for visualizing information nets in three dimensions. *ACM Transactions on Graphics*, 15(2), 121–140.

[12] Wettel, R., & Lanza, M. (2007). Visualizing software systems as cities. *Proceedings of VISSOFT*, 92–99.